Problem Set 6

Name: Nicolas Iskandar

Collaborators: None

Problem 6-1.

(a) Relaxation order is unique for both algorithms.

DAG Relaxation: $(a,b),\ (a,d),\ (d,b),\ (d,e),\ (b,c),\ (b,e),\ (b,f),\ (e,f),\ (f,c)$

Dijkstra: $(a,b),\ (a,d),\ (b,c),\ (b,e),\ (b,f),\ (f,c),\ (e,f),\ (d,b),\ (d,e)$

(b)

v	a	b	c	d	e	f
$\delta(a,v)$	0	2	3	6	5	4

Problem 6-2. **Idea.** Standard Bellman–Ford needs $|V|-1$ rounds of relaxing every edge, because a shortest simple path can use up to $|V|-1$ edges. Here we are guaranteed every vertex has some shortest path using at most k edges, so we only need k rounds instead of $|V|-1$.

Key fact. After round i of Bellman–Ford (relaxing every edge i times), the distance array holds the correct shortest-path distance to each vertex *restricted to paths of at most i edges*. Round 1 gives the best 1-edge paths, round 2 the best 2-edge paths, etc. (Formally this corresponds to layer i of the duplicated DAG used to prove Bellman–Ford's correctness; a shared distance array updated in place implements this without building the layers explicitly.)

Algorithm.

1.Initialize $\delta(s)=0$, $\delta(v)=\infty$ for all other v. $O(V)$

2.Repeat k times: relax every edge once (one pass over E). $O(kE)$

3.Return the distance array.

Correctness. Each round extends "best path using $\leq i$ edges" to "best path using $\leq i+1$ edges." Since every vertex has an optimal path with $\leq k$ edges by hypothesis, round k is guaranteed to have found it; further rounds cannot improve anything.

Running time. $O(V)$ for initialization plus k rounds each costing $O(E)$ gives $O(V+kE)$ total.

Problem 6-3. Why elevation changes break a single DAG. Detonation lowers the elevation of every clearing in C_D, which can *flip* which direction is downhill on any slope touching one of those clearings. A single graph cannot represent both directions of the same edge simultaneously, so a plain shortest-path computation over one graph is insufficient.

Construction. Build two copies of the clearing graph:

- G_1: one node per clearing using original elevations e_i; a directed edge for each slope pointing toward the lower clearing, weighted by slope length (drop non-monotonic/flat slopes).
- G_2: identical construction, but using post-detonation elevations e_i' for $c_i \in C_D$.
- One zero-weight edge $D_1 \to D_2$ (detonating: instantaneous, costs no distance). No edge goes from G_2 back to G_1, detonation is permanent, so this keeps the combined graph acyclic.
- A supernode T with zero-weight edges in from S_1 and S_2, merging "reach S without detonating" and "reach S after detonating" into a single target.

Run DAG Relaxation from L_1 to T on the combined graph G; the shortest path automatically chooses whichever strategy (detonate or not) is cheaper.

Running time. G_1, G_2 each have $O(n)$ vertices/edges, so G has $O(n)$ vertices and edges total. DAG Relaxation runs in $O(V + E) = O(n)$.

Problem 6-4. Reframing. Build a directed weighted graph G: one vertex per location, one edge per transition weighted by its work w_{ij}. The field is conservative iff every cycle in G has total weight exactly 0.

Key idea. Bellman–Ford detects negative-weight cycles as a byproduct of computing shortest paths. Repurpose this as the actual detection tool:

- Run Bellman–Ford once on the original weights, catches any *negative*-weight cycle.
- Negate every edge weight and run Bellman–Ford again, a positive-weight cycle in the original graph becomes negative-weight in the negated graph (weights, and hence cycle totals, flip sign under negation; zero-weight cycles stay zero), so this run catches any *positive*-weight cycle.

If either run reports a negative cycle, the field is not conservative; otherwise every cycle has weight 0 and the field is conservative.

Supernode. Add vertex s with a zero-weight edge to every other vertex (one-directional, out of s only, adds no new cycles). This guarantees every vertex, and hence every cycle, is reachable from s, so a single Bellman–Ford run from s suffices to catch a negative cycle anywhere in the graph.

Running time. $V = O(n)$, $E = O(n)$ after adding s and its edges. Each Bellman–Ford run costs $O(VE) = O(n^2)$; two runs are still $O(n^2)$.

Problem 6-5. Why plain Dijkstra fails. Whether a road is usable depends on current gas level (uphill travel costs gas and the tank must stay strictly positive), so a single vertex per intersection cannot capture enough state: "at A with 5 gas" and "at A with 1 gas" allow different future moves.

State-space construction. Make one vertex v_i per (intersection v, gas level i), for $i = 1, \ldots, g$. For each road $\{u, v\}$ with driving time $t(u, v)$ and $e(u) \leq e(v)$:

- *Downhill/flat* $(v_i \to u_i)$ for every i, weight $t(u, v)$ (free in gas).
- *Uphill*, costing $\ell = e(v) - e(u)$ gas: edge $(u_i \to v_{i-\ell})$ only for $i > \ell$, weight $t(u, v)$, this is exactly how "tank stays strictly positive" is enforced, by omitting edges that would violate it.
- *Refuel* at a station v: edge $(v_i \to v_{i+1})$, weight t_G, for $i < g$. Refueling by k units in one action costs $k \cdot t_G$, identical to taking k such edges in a row, so no information is lost by only modeling unit refills.

All weights are non-negative, and the graph may contain cycles, so run Dijkstra (not DAG Relaxation) once from s_g (start full). The answer is $\min_i \delta(s_g, t_i)$.

Running time. Average degree $< 5 \Rightarrow r = O(n)$; since $g < n$, the blown-up graph has $O(n^2)$ vertices and edges, built in $O(n^2)$ time. Dijkstra with a binary heap costs $O(E \log V) = O(n^2 \log n)$, dominating the final $O(n)$-time scan over the t_i. Total: $O(n^2 \log n)$.